JACK VANLIGHTLY

October 8, 2025

Beyond Indexes: How Open Table Formats Optimize Query Performance

Jack Vanlightly · Data

My career in data started as a SQL Server performance specialist, which meant I was deep into the nuances of indexes, locking and blocking, execution plan analysis and query design. These days I'm more in the world of the open table format such as Apache Iceberg. Having learned the internals of both transactional and analytical database systems, I find the use of the word "index" interesting as they mean very different things to different systems.

I see the term "index" used loosely when discussing open table format performance, both in their current designs and in speculation about future features that might make it into their specs. But what actually counts as an index in this world?

Some formats, like Apache Hudi, do maintain record-level indexes such as, primary-key-to-filegroup maps that enable upserts and deletes to be directed efficiently to the right filegroup in order to support primary key tables. But they don't help accelerate read performance across arbitrary predicates like the secondary indexes we rely on in OLTP databases.

Traditional secondary indexes (like the B-trees used in relational databases) don't exist in Iceberg, Delta Lake, or even Hudi. But why? Can't we solve some performance issues if we just added secondary indexes to the Iceberg spec?

The short answer is: “*no and it's complicated*”. There are real and practical reasons why the answer isn’t just "*we haven't gotten around to it yet*."

The aim of the post is to understand:

1. Why we can’t just add more secondary indexes to speed up analytical queries like we can often do in RDBMS databases.
2. How the analytics workload drives the design of the table format in terms of core structure and auxiliary files (and how that differs to the RDBMS).

One core thing I’m going to try to convince you of in this post is that whether your table is a traditional RDBMS or Iceberg, read performance comes down to reducing the amount of IO performed, which in turn comes down to:

1. How we organize the data itself.
2. The use of auxiliary data structures over that data organization.

The key point is that the workload determines how you organize that data and what kind of auxiliary data structures are used. We’ll start by understanding indexing in the RDBMS and then switch over and contrast that to how open table formats such as Iceberg work.

Data organization, indexes and auxiliary structures in traditional relational databases

At its core, an index speeds up queries by reducing the amount of IO required to execute the query. If a query needs one row but has to scan the entire table, then that will be really slow if the table is large. An index in

an RDBMS is a type of B-tree and provides two main access patterns: a *seek* which traverses the tree looking for a specific row or a *scan* which iterates over the whole tree, or a range of the tree.

We can broadly categorize indexes into two types in the traditional RDBMS:

- **Clustered index**. The index and the data aren't separate, they are clustered together. The table is the clustered index. Like books on a library shelf arranged alphabetically by author. The ordering is the collection itself; finding "Tolkien" means walking directly to the T section, because the data is stored in that order.
- **Non-clustered index**. The index is a separate (smaller) structure that points to the actual table rows. A secondary index is an example of this. Like in the library there may be a computer that allows you to search for books, and which tells you the shelf locations.

Then we have table statistics such as estimated cardinalities and histograms which, as we'll see further down, become very useful for the query optimizer to select an efficient execution plan.

Let's look at these three things: the clustered index, non-clustered index and column statistics. I could add heap-based tables and a bunch of other things but let's try and be brief.

The RDBMS table itself: aka the clustered index

Let's imagine we have a User table, with UserId as the primary key, with multiple columns including FirstName, LastName, Country and Nationality. The table is a clustered (B-tree) index, which is sorted by UserId.

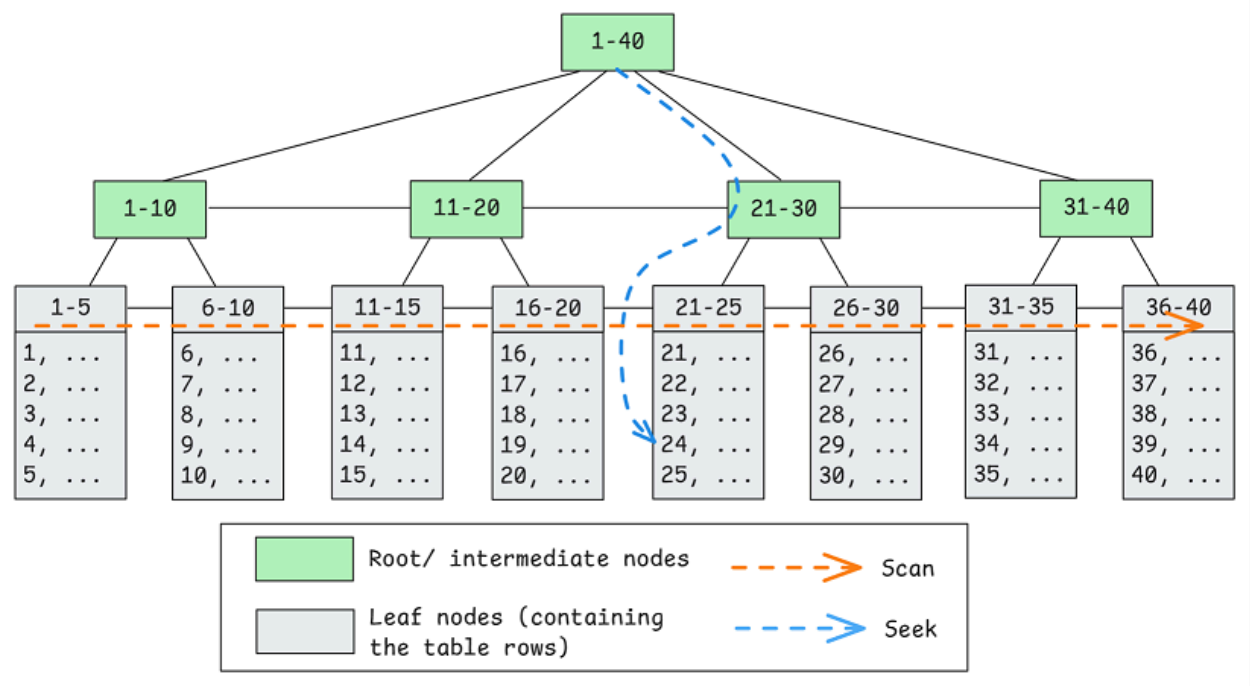

Fig 1. The clustered index with a primary key of UserId as an auto-incrementing int.

If you query the table by the primary key, the database engine can do a B-tree traversal and rapidly find the location of the desired row. For example, if I run *SELECT * FROM User WHERE UserId = 100;* then the database will do a clustered index seek, which means it traverses the clustered index by key, quickly finding the page which hosts the desired row.

When rows are inserted and deleted, they must be added to the B-tree which may require new data pages, page splits and page merges when data pages become full or too empty. Fragmentation can occur over time such that the B-tree data pages are not laid out sequentially on disk, so index rebuilds and defragmentation can be needed.

All this tree management is expensive but the benefits are worth it because in an OLTP workload, queries are overwhelmingly about finding or updating a single row, or a very small set of rows, quickly. A B-tree sorted by the primary key makes this possible: the cost of a lookup is only

O(log n) regardless of table size, and once the leaf page is reached, the row is right there. That means whether the table has a thousand rows or a hundred million, a clustered index seek for *UserId = 18764389* takes just a handful of page reads. The same applies to joins: if another table has a foreign key reference to UserId, the database can seek into the clustered index for each match, making primary-key joins fast and predictable. Inserts and deletes may cause page splits or merges, but the tree structure ensures the data remains in key order, so seeks stay efficient.

So far so good, but what about when you want to query the Users table by country or last name? That leads us to secondary (non-clustered) indexes.

The secondary, non-clustered index

My application also needs to be able to execute queries with predicates based on Country and LastName. A where clause on either of these two columns would result in a full table scan (clustered index scan) to find all rows which match that predicate. The reason is that the B-tree intermediate nodes simply contain no information about the Country and LastName columns.

To cater for queries that do not use the primary key, we could use one or more secondary indexes, which are separate B-trees which map specific non-PK columns to rows in the clustered index. In SQL Server, a non-clustered index that points to a clustered index stores the indexed column(s) value and clustering key, aka, the primary key of the match.

I could create one secondary index on the column ‘LastName’ and another on ‘Country’.

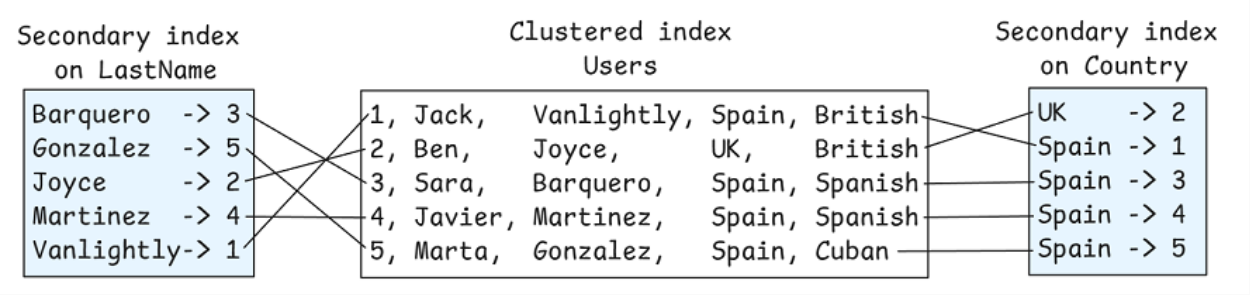

Fig 2. Secondary indexes map columns to the clustering key (primary key) so the query engine can perform a clustered index seek. Note that in small tables, secondary indexes are not used as it's quicker to scan the table.

My *SELECT * FROM Users WHERE LastName = 'Martinez'* will use a *lookup* operation, which is a:

1. Seek on the secondary index B-tree to obtain the clustering key (PK) of the matching Martinez row.
2. A seek on the clustered index (the table itself) using the clustering key (PK) to retrieve the full row.

Selectivity is important here. If the secondary index scan returns a small number of rows, this random IO of multiple index seeks is worth it, but if the scan returns a large number of rows, it might actually be quicker to perform a clustered index scan (full table scan).

Imagine if the table had 1 million rows and the distribution of the Country column matched the figure above, that is to say roughly 800,000 rows had 'Spain' in the Country column. 800,000 lookups would be far less efficient than one table scan. The degree of selectivity where either lookups from a secondary index vs a clustered index scan can be impacted depending on whether the database is running on a HDD or SSD, with SSDs tolerating more random IO than a HDD. It is up to the query optimizer to decide.

Secondary indexes are ideal for high selectivity queries, but also queries that can be serviced by the secondary index alone. For example, if

I only wanted to perform a count aggregation on Country, then the database could scan the Country secondary index, counting the number of rows with each country and never need to touch the much larger clustered index. We've successfully reduced the amount of IO to service the query and therefore sped it up.

But we can also add "covering columns" to secondary indexes. Imagine the query *SELECT Nationality FROM User WHERE Country = 'Spain'*. If the user table is really wide and really large, then we can speed up the query by adding the Nationality column to the Country secondary index as a *covering column*. The query optimizer will see that the Country secondary index includes everything it needs and decides not to touch the clustered index at all. Covering columns make the index larger which is bad, but can remove the need to seek the clustered index which is good.

These secondary indexes trade off read performance for space, write cost and maintenance cost overhead. Each write must update the secondary indexes synchronously, and sometimes these indexes get fragmented and need rebuilding. For an OLTP database, this cost can be worth it, and the right secondary index or indexes on a table can help support diverse queries on the same table.

Column statistics

Just because I created a secondary index doesn't mean the query optimizer will use it. Likewise, just because a secondary index exists doesn't mean the query optimizer *should* choose it either. I have seen SQL Server get hammered because of incorrect secondary index usage. The optimizer believed the secondary index would help so it chose the "lookup" strategy, but it turned out that there were tens of thousands of matching rows (instead of the handful predicted), leading to tens of thousands of individual clustered index seeks. This can turn into millions of seeks in queries if the Users table exists in several joins.

Table statistics, such as cardinalities and histograms are very useful here to help the optimizer decide which strategy to use. For example, if every row contains 'Spain' for country, the cardinality estimate will tell the optimizer not to use the secondary index for a *SELECT * FROM Users WHERE Country = 'Spain'* query as the selectivity is not low enough. Best to do a table scan.

Histograms are useful when the cardinality is skewed, such that some values have large numbers of rows while others have relatively few.

RDBMS summary

There is, of course, far more to write about this topic, way more nuances on indexing, plus alternatives such as heap-based tables, other trees such as LSM trees, and alternative data models such as documents and graphs. But for the purpose of this post, we keep our mental model to the following:

- **Workload**: optimized for point lookups, short ranges, joins of a small number of rows, and frequent writes/updates. Queries often touch a handful of rows, not millions.
- **Data organization**: tables are row-based; each page holds complete rows, making single-row access efficient.
- **Clustered index**: the table itself is a B-tree sorted by the primary key; lookups and joins by key are fast and predictable (O(log n)).
- **Secondary (non-clustered) index**: separate B-trees that map other columns to rows in the clustered index. Useful for high-selectivity predicates and covering indexes, but costly to maintain.
- **Statistics**: cardinalities and histograms guide the optimizer to choose between using an index or scanning the table.

- **Trade-off**: indexes cut read I/O but add write and maintenance overhead. In OLTP this is worth it, since selective access dominates.

Perhaps the main point for this post is that secondary indexes allow for one table to support diverse queries. This is something that we cannot easily say of the open table formats as we'll see next.

Data organization, indexes and auxiliary structures in the open table formats

Analytical systems have a completely different workload from OLTP. Instead of reading a handful of rows or updating individual records, queries in the warehouse or lakehouse typically scan millions or even billions of rows, aggregating or joining them to answer analytical questions. Where row-based storage in a sorted B-tree structure makes sense for OLTP, it is not efficient for analytical workloads. So instead, data is stored in a columnar format in large contiguous blocks with a looser global structure.

Secondary indexes aren't efficient either. Jumping from index to row for millions of matches would already be massively inefficient on a file system, but given OTFs are hosted on object storage, secondary indexing becomes even less useful. So the analytical workload makes row-based B-trees and secondary indexes the wrong tool for the job.

Columnar systems flipped the model: store data in **columns rather than rows**, grouped into large contiguous blocks. Instead of reducing IO via B-tree traversal by primary key, or pointer-chasing through secondary indexes, IO reduction comes from effective **data skipping** during table scans. The columnar storage allows for whole columns to be skipped and depending on a few other factors, entire files can be skipped as well. Most data skipping happens during the planning phase, where the execution engine decides which files need to be scanned, and is often referred to as

pruning. There are different types of pruning and as well as different levels of pruning effectiveness which all depend on how the data is organized, what metadata exists and any other auxiliary search optimization data structures that might exist.

Let's look at the main ingredients for effective pruning: the table structure itself and the auxiliary structures.

The table structure: metadata + data files

Let's focus on Iceberg for now to keep ourselves grounded in a specific table format design. I did an [in-depth description of Iceberg internals] based on v2 which is still valid today, if you want to dive in deep.

To keep things simple, we'll look at the organization of an Iceberg table at one point in time.

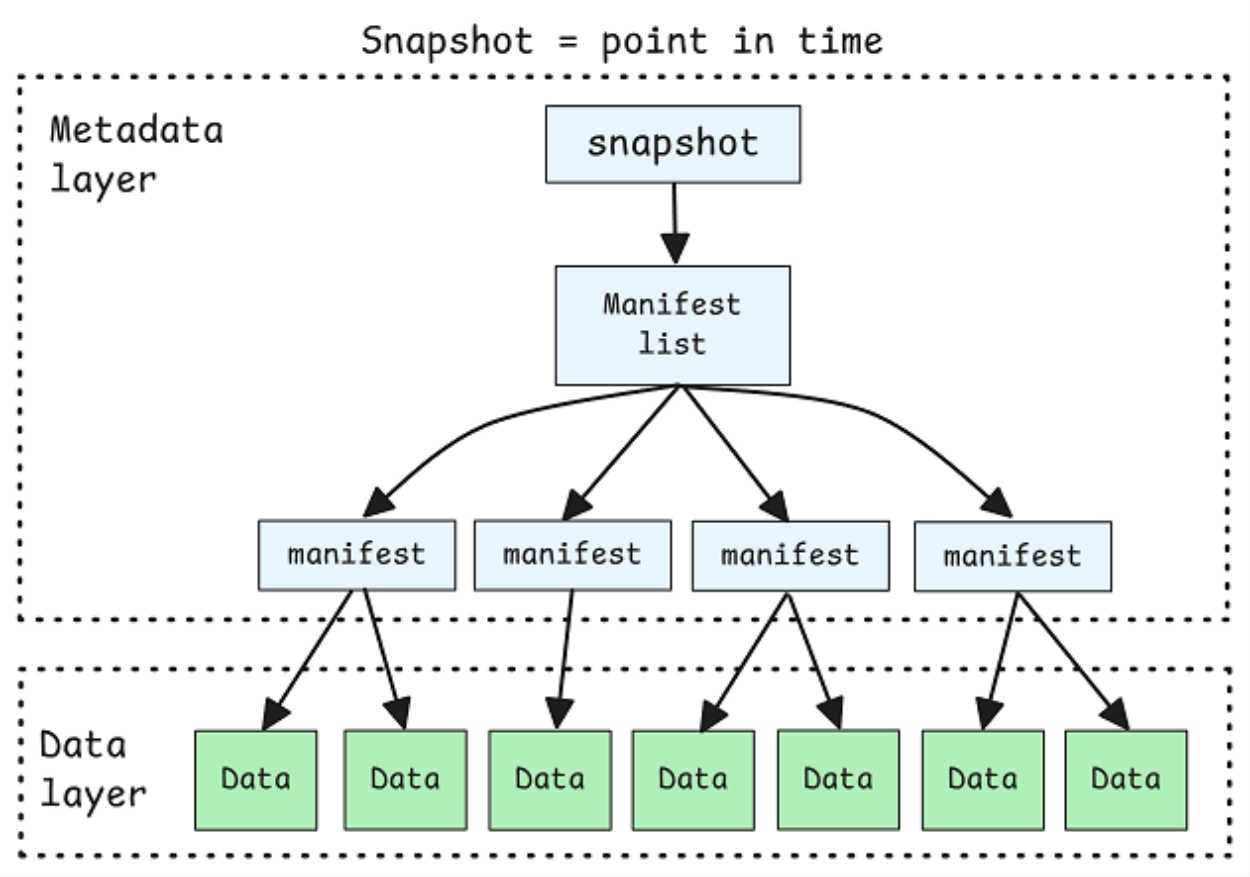

Fig 3. An Iceberg table is queried based on a single snapshot, which forms a tree of metadata and data files.

An Iceberg table also forms a tree, but it looks very different from the sorted B-tree of a clustered index. At the root is the metadata file, which records the current snapshot. That snapshot points to a manifest list, which in turn points to many manifest files, each of which references a set of data files. The data files are immutable Parquet (or ORC) files. A clustered index is inherently ordered by its key, but an Iceberg table's data files have no intrinsic order, they're just a catalogued set of Parquet files. Given this unordered set of files managed by metadata, performance comes from effective pruning.

Effective pruning comes down to data locality that aligns with your queries. If I query for rows with *Country = 'Spain'*, but the Spanish data is spread across all the files, then the query will have to scan every file. But if all the Spanish data is grouped into one small subset of the entire data set, then the query only has to read that subset, speeding it up.

We can split the discussion of effective pruning into:

- **The physical layout of the data**. How well the data locality and sorting of the table aligns with its queries. Also the size and quantity of the data files.
- **Metadata available to the query planner**. The metadata and auxiliary data structures used by the query planner (and even during the execution phase in some cases) to prune files and even blocks within files.

The physical layout of the data

An Iceberg table achieves data locality through multiple layers of grouping, both across files and within files (via Parquet or ORC):

- **Partitioning**: groups related rows together across a subset of data files, so queries can target only the relevant partitions.
- **Sorting**: orders rows within each partition so that values that are logically close are also stored physically close together within the same file or even the same row group inside a file.

Partitioning is the first major grouping lever available. It determines how data files are organized into data files dividing the table into logical partitions based on one or more columns, such as a date (or month/year), a region, etc. All rows sharing the same partition key values are written into the same directory or group of files. This creates data locality such that when a query includes a filter on the partition column (for example, *WHERE EventDate = '2025-10-01'*), the engine can identify exactly which partitions contain relevant data and ignore the rest. This process is known as **partition pruning**, and it allows the engine to skip scanning large portions of the dataset entirely. The more closely the partitioning aligns with typical query filters, the more effective pruning becomes.

Great care must be taken with the choice of partition key as the number of partitions impacts the number of data files (with many small files causing a performance issue).

Within each partition, the data files have no inherent order by themselves. This is where **sort order** becomes important. Sorting defines how rows are organized within each partition, typically by one or more columns that are frequently used in filter predicates or range queries. For example, if queries often filter by EventTime or Country, sorting by those columns ensures that rows with similar values are stored close together.

Sorting improves data locality both within files and across files:

- Within files, sorting determines how rows are organized as data is written, ensuring that similar values are clustered together. This groups similar data together (according to sort key) in Parquet row groups and makes row group level pruning more effective.
- Sort order can influence grouping across data files during compaction. When files are written in a consistent sort order, each file tends to represent a narrow slice of the sorted key space. For example, the rows of Spain might slot into a single file or a handful of files when Country is a sort key.

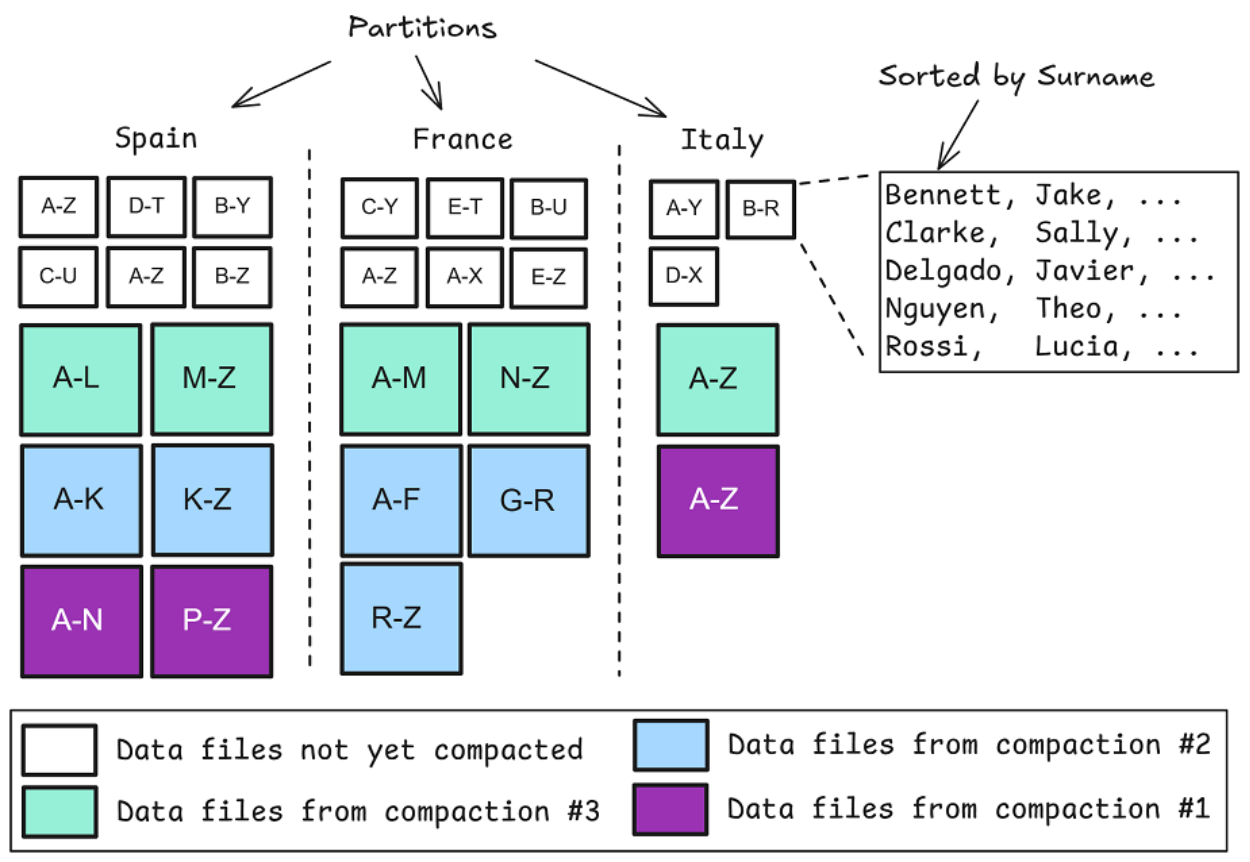

Fig 4. A table with three partitions by Country and sort order by Surname. Sort order is preserved, but less effective on newly written small files. Compactions can sort rows of a large group of files into fewer larger files with ordering preserved across them.

Partitioning is a strong, structural form of grouping that is enforced at all times. Once data is written into partitions, those boundaries are fixed and

guaranteed, making partition pruning a reliable and predictable optimization. Sort order, on the other hand, is a more dynamic process that becomes most effective after compaction. Each write operation typically sorts its own batch of data before creating new files, but this sorting happens in isolation. Across multiple writes, even if all use the same declared sort order, there's no partition-wide reordering of existing data. As a result, files within a partition may each be internally sorted but not well ordered relative to one another. Compaction resolves this by rewriting many files into fewer, larger ones while enforcing a consistent sort order across them. Only then does the sort order become a powerful performance optimization for data skipping.

Delta Lake has similar features, with partitioning and sorting. Delta supports Z-ordering, which is a multidimensional clustering technique. Instead of sorting by a single column, Z-order interleaves the bits of multiple column values (for example, Country, Nationality, and EventDate) into a single composite key that preserves spatial locality. This means that rows with similar combinations of those column values are likely to be stored close together in a file, even if no single column is globally sorted. Z-ordering is particularly effective for queries that filter on multiple dimensions simultaneously.

How rows are organized across files and within files impacts pruning massively. Next let's look at how metadata and auxiliary data structures help the query planner make use of that organization.

Metadata and auxiliary data structures

Column statistics

Just like in the RDBMS, column statistics are critically important, however, for the open table format, they are even more important.

Column statistics are the primary method that query engines use to prune data files and row groups within files.

In Iceberg we have two sources of column statistics:

- **Metadata files**: Manifest files list a set of data files along with per-column min/max values. Query engines can use these column stats to skip entire files during the planning phase based on the filter predicates.
- **Data files**: A Parquet file is divided into row groups, each containing a few thousand to millions of rows stored column by column. For every column in a row group, Parquet records min/max statistics, and query engines can use these to skip entire row groups when the filter predicate falls outside the recorded range.

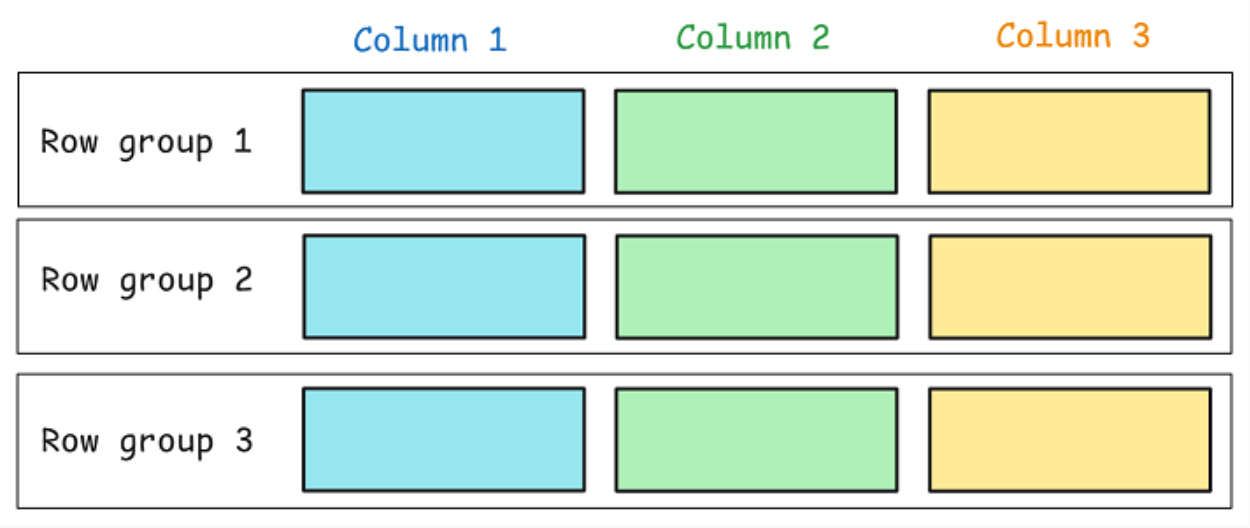

Fig 4. Parquet splits up the data into one or more row groups, and within each row group, each column is stored contiguously.

Min/max column statistics are a mirror of how the data is laid out. When data is clustered/sorted by a column, that column's statistics reflect that structure: each file or row group will capture a narrow slice of the column's value range. For example, if data is sorted by Country, one

file might record a min/max of ["France"–"Germany"] while another covers ["Spain"–"Sweden"]. These tight ranges make pruning highly effective. But when data isn't organized by that column and all countries are mixed randomly across files, the statistics show it. Each file's min/max range becomes broad, often spanning most of the column's domain. The result is poor selectivity and less effective pruning, because the query planner can't confidently skip any files.

Simply put, column statistics are a low cost tool for the query planner to understand the data layout and plan accordingly.

Other auxiliary data structures

There are other auxiliary structures that enhance pruning and search efficiency, beyond min/max statistics.

Bloom filters are one such mechanism. A Bloom filter is a compact probabilistic data structure that can quickly test whether a value might exist in a data file (or row group). If the Bloom filter says "no," the engine can skip reading that section entirely; if it says "maybe," the data must still be scanned. Iceberg and Delta can store Bloom filters at the file or row-group level, providing an additional layer of pruning beyond simple min/max range checks. Bloom filters are especially useful on columns that are not part of the sort order (so min/max stats are less effective) or exact match predicates.

Some systems also maintain custom or proprietary sidecar files that act as lightweight search/data skipping structures. These can include:

- More fine-grained statistics files, such as histograms.
- Table level Bloom filters (Hudi).
- Primary-key-to-file index to support primary key tables (Hudi, Paimon). Upserts and deletes can be directed to specific files via the

index.

Materialized views

Other techniques involve creating optimized materialized projections, known as **materialized views**. These are precomputed datasets derived from base tables, often filtered, aggregated, or reorganized to match common query patterns. The idea is to store data in the exact shape queries need, so that the engine can answer those queries without repeatedly scanning or aggregating the raw data. They effectively trade storage and maintenance cost for speed, just like secondary indexes do in the RDBMS. Some engines implement these projections transparently, using them automatically when a query can be satisfied by a precomputed result, just like an RDBMS will choose a secondary index if the planner decides it is more efficient. This can be much more powerful than just a materialized view as a separate table. One example is Dremio's Reflections, though there may already be other instances of this in the wild among the data platforms.

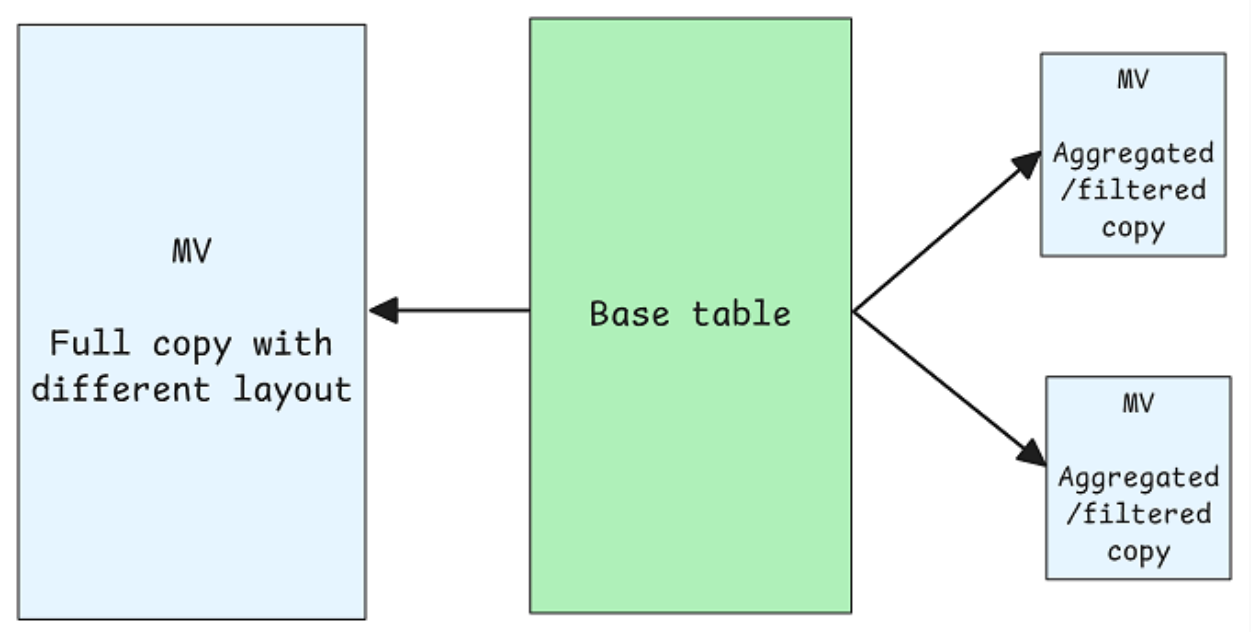

Fig 5. Materialized views help support diverse queries that the base table, with its one layout, cannot. Making such materialized views transparent to the user, by

allowing the query planner to use the MV instead of the base table is a powerful approach, akin to the use of secondary indexes.

Open table format summary

- Open table formats like Iceberg, Delta, and Hudi store data in immutable, columnar files, optimized for large analytical scans.
- Query performance depends on data skipping (pruning), which is the ability to avoid reading irrelevant files or row groups based on metadata.
- Pruning effectiveness depends on data layout.
- Data layout levers:
    - Partitioning provides strong physical grouping across files, enabling efficient partition pruning when filters match partition keys.
    - Sorting improves data locality within partitions, tightening column value ranges and enhancing row-group-level pruning.
    - Compaction consolidates small files and enforces consistent sort order, making pruning more effective (and reducing the small file cost that partitioning can sometimes introduce).
    - Z-ordering (Delta) and Liquid Clustering (Databricks) extend sorting to multi-dimensional and adaptive clustering strategies.
- Column statistics in Iceberg manifest files and Parquet row groups drive pruning by recording min/max values per column. The statistics reflect the physical layout.
- Bloom filters add another layer of pruning, especially for unsorted columns and exact match predicates.
- Some systems maintain sidecar indexes such as histograms or primary-key-to-file maps for faster lookups (e.g., Hudi, Paimon).

- Materialized views and precomputed projections further accelerate queries by storing data in the shape of common query patterns (e.g., Dremio Reflections). These require some data duplication and data maintenance, and are the closest equivalent (in spirit) to the secondary index of an RDBMS.

Analytics data modeling

The physical layout of the individual table isn't the whole story. Data modeling plays just as critical a role. In a star schema, queries follow highly predictable patterns: they filter on small dimension tables and join to a large fact table on foreign keys. That layout makes diverse analytical queries fast without the need for secondary indexes. The dimensions are small enough to be read in full (and often cached), so indexing them provides no real gain.

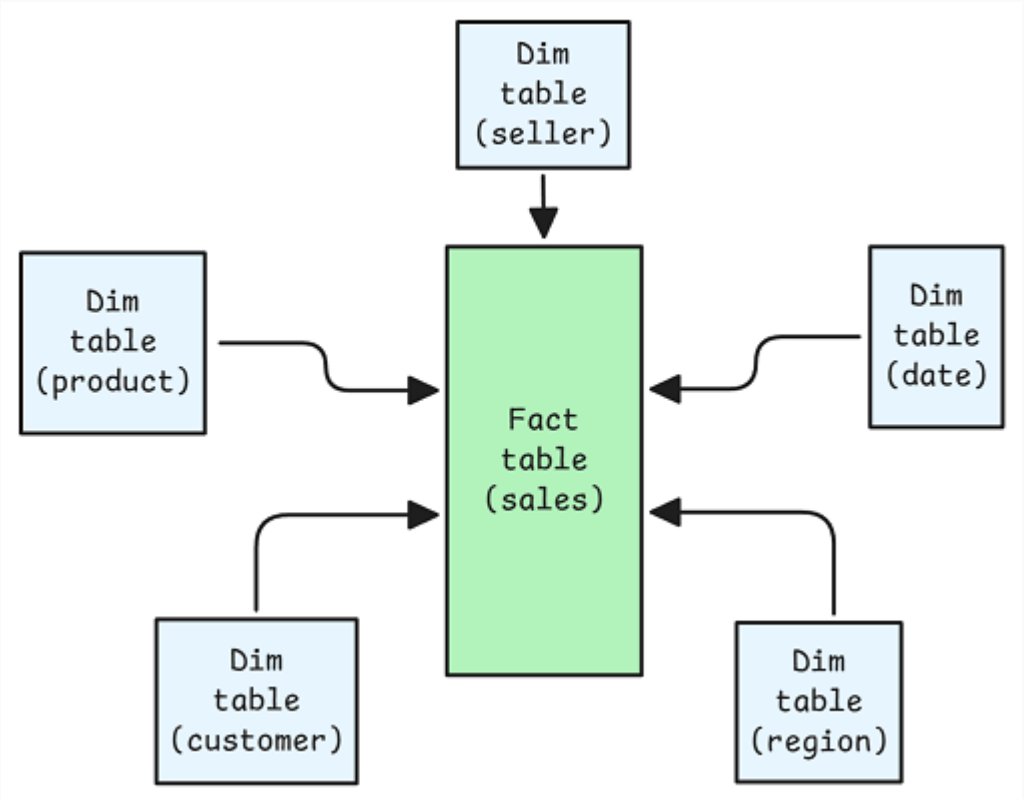

Fig 6. The central fact table contains information such as sales, with foreign keys to the dimension tables and facts such as sale amount, quantity, etc. Dimensions may include products, customers,

> *vendors, dates and so on. A product dimension will have a ProductId and a number of columns describing aspects of the product such as manufacturer, SKU, etc.*

In an Iceberg-based star schema, partitioning should follow coarse-grained dimensions like time, such as *days(sale_ts)* or *months(sale_ts)* or by region, rather than high-cardinality surrogate IDs which would fragment the table too much. However, Iceberg's bucketing partition option can still turn surrogate IDs into large boundaries, such as *bucket(64, customer_id)*. Sorting within partitions by the most selective keys like customer_id, product_id, or sale_ts makes Iceberg's column statistics far more effective for skipping data in the large fact table. The goal is to make the table's physical structure mirror the star schema's logical access paths, so Iceberg's metadata and statistics work efficiently.

The star-schema can serve queries on various predicates efficiently, without needing ad hoc indexes for every possible predicate. The tradeoff is the complexity and cost of transforming data into this star schema (or other scheme).

Final thoughts

If you squint, the clustered index of a relational database and the Parquet and metadata layers of an open table format such as Iceberg share a common goal: *minimize the amount of data scanned*. Both rely on structure and layout to guide access, the difference is that instead of maintaining B-tree structures, OTFs lean on looser data layout and lightweight metadata to guide search (pruning being a search optimization). This makes sense for analytical workloads where queries often scan millions of rows: streaming large contiguous blocks is cheap, but random pointer chasing and maintaining massive B-trees is not. Secondary indexes, so valuable in OLTP, add little to data warehousing.

Analytical queries don’t pluck out individual rows, they aggregate, filter, and join across millions.

Ultimately, in open table formats, layout is king. Here, performance comes from layout (partitioning, sorting, and compaction) which determines how efficiently engines can skip data, while modeling, such as star or snowflake schemas, make analytical queries more efficient for slicing and dicing. Unlike a B-tree, order in Iceberg or Delta is incidental, shaped by ongoing maintenance. Tables drift out of shape as their layout diverges from their queries, and keeping them in tune is what truly preserves performance.

Since the OTF table data exists only once, its data layout must reflect its dominant queries. You can’t sort or cluster the table twice without making a copy of it. But it turns out that copying, in the form of materialized views, is a valuable strategy for supporting diverse queries over the same table, as exemplified by Dremio Reflections. These make the same cost trade offs as secondary indexes: space and maintenance for read speed.

Looking ahead, the open table formats will likely evolve by formalizing more performance-oriented metadata. The core specs may expand to include more diverse standardized sidecar files such as richer statistics so engines can make more effective pruning decisions. But secondary data structures like materialized views are probably destined to remain platform-specific. These are alternate physical representations of data itself (not just metadata), with their own lifecycles and maintenance processes. The open table formats focus on representing one canonical table layout clearly and portably. Everything beyond that such as automated projections and adaptive clustering is where engines will continue to differentiate. In that sense, the future of the OTFs lies not in embedding more sophisticated logic, but in providing the hooks for smarter systems to build on top of them.

What, then, is an “index” in this world? The closest thing we have is a Bloom filter, with column statistics and secondary materialized views serving the same spirit of guiding the query planner toward efficiency. Traditional secondary indexes are nowhere to be found, and calling an optimized Iceberg table an index (in clustered index sense) stretches the term too far. But language tends to follow utility, and for lack of a better term, “index” will probably keep standing in for the entire ecosystem of metadata, statistics, and other pruning structures that make these systems more efficient.

In the future I’d love to dive into how Hybrid Transactional Analytical Processing (HTAP) systems deal with the fundamentally different nature of transactional and analytical queries. But that is for a different post.

Thanks for reading.

ps, if you want to learn more about table format internals, I’ve written extensively on it: The ultimate guide to table format internals - all my writing so far